



DOCUMENTATION

TECHNIQUE

Introduction

Ce dossier présente la documentation technique de l'application EcoRide, un projet complet visant à développer une application web de covoiturage éco responsable.

Il détaille les choix technologiques retenus, la configuration de l'environnement de travail, la modélisation de l'architecture via des diagrammes UML, ainsi que les principales étapes de déploiement.

Cette documentation a pour objectif de démontrer ma capacité de comprendre un besoin client, choisir une stack adaptée, concevoir, développer et mettre en production une application web et web mobile dans des conditions proches du monde professionnel.

1. Réflexions initiales technologiques

Avant de commencer le développement de l'application EcoRide, il était essentiel d'analyser les besoins techniques et de sélectionner les technologies les plus adaptées. Aucune stack technique n'était imposée, il a donc fallu faire un choix en tenant compte des critères suivant :

- Performance
- Sécurité
- Maintenance
- Adaptation au besoin métier
- Compatibilité avec le déploiement en ligne
- Expérience utilisateur
- Impact environnemental

Et bien sûr ma propre maîtrise des technologies. Voici donc la stack technique que j'ai retenu :

a. Front-end : Bootstrap (HTML, CSS, JS)

Pour la partie front-end de l'application EcoRide, j'ai choisi Bootstrap, en premier lieu pour son **responsive design**. C'est l'un des principaux avantages de Bootstrap et la raison pour laquelle je l'ai priorisé. Sa grille fluide à 12 colonnes et ses composants adaptatifs garantissent que l'interface s'affiche de façon optimale sur tous les types d'écrans, sans avoir à gérer de nombreux media queries complexes, ce qui réduit le risque d'oublis.

Grâce à ses composants prêts à l'emploi (cartes, formulaires...), Bootstrap m'a permis de maintenir une charte graphique homogène et donc une harmonie visuelle. De plus, Bootstrap intègre déjà de nombreuses recommandations RGAA, telles que les contrastes suffisants et des attributs ARIA sur ses composants interactifs. Cela a facilité l'inclusivité de mon application.

En complément, j'ai intégré JavaScript pour **dynamiser** certaines fonctionnalités du projet, notamment les filtres sur les résultats de trajets.

Après une première recherche, l'utilisateur peut en effet affiner les trajets (prix et durée maximum, note minimum ou voyage écologique). Grâce à JavaScript, ces filtres s'appliquent directement dans le navigateur, ce qui offre une expérience fluide et réactive sans rechargement de page.

b. Back-end : Symfony (PHP)

Après plusieurs semaines de formations et de pratiques du langage PHP, j'ai choisi d'utiliser Symfony comme framework pour plusieurs raisons.

Tout d'abord, pour son **cadre MVC** structuré. En effet, Symfony propose une architecture Modèle-Vue-Contrôleur qui permet d'organiser le code proprement et de bien séparer la logique métier, les contrôleurs et les vues. Cela facilite considérablement la maintenance et l'évolution du projet. Pour une développeuse débutante, c'est un cadre rassurant qui guide naturellement vers les bonnes pratiques.

Un autre atout rassurant est la **sécurité** intégrée, puisque Symfony fournit dès l'installation des fonctionnalités avancées pour sécuriser l'application, comme la protection CSRF, la gestion de l'authentification et des sessions ou encore le hashage des mots de passe avec bcrypt intégré. Symfony permet également la possibilité de gérer simplement l'accès selon les profils (utilisateur, employé, admin).

Symfony est livré avec **Doctrine**, un ORM puissant qui simplifie la gestion des données relationnelles avec MySQL. Il permet de manipuler les entités PHP directement, tout en s'occupant des requêtes SQL et de la sécurité (requêtes préparées, transactions, relations entre les tables).

Enfin, Symfony dispose d'un grand nombre de **bundles** et d'extensions qui permettent d'ajouter facilement des fonctionnalités comme la gestion des mails avec **Mailer**. Sa communauté est très active et la documentation est complète, ce qui est souvent d'une aide précieuse.

En choisissant Symfony, ma volonté était aussi de me former directement à un framework reconnu en entreprise, ce qui pourra être un atout majeur pour la suite de mon parcours professionnel.

c. Base de donnée relationnelle : MySQL

Pour gérer les données principales de l'application EcoRide (utilisateurs, trajets, réservations, avis, transactions...), j'ai choisi d'utiliser MySQL, qui s'adapte à des données fortement structurées et liées.

Dans EcoRide, les données sont fortement interconnectées : un utilisateur peut être passager, chauffeur, employé ou admin, un trajet appartient à un chauffeur et référence un véhicule, une réservation relie un passager à un trajet, un avis est lié à une réservation, validé par un employé, etc.

Ce type de données hautement relationnelles est parfaitement géré par MySQL grâce à ses contraintes d'intégrité (clés étrangères, jointures).

J'utilise également Doctrine, l'ORM intégré à Symfony. En effet, suite à mes recherches, j'ai compris que l'utilisation de Doctrine était préférable à l'écriture de requêtes SQL natives. En effet, Doctrine repose sur des requêtes paramétrées, ce qui renforce la sécurité en protégeant l'application contre les injections SQL, conformément aux bonnes pratiques recommandées par l'OWASP.

(Toutes mes requêtes ne sont pas encore réécrites, mais c'est un travail en cours.)

d. Base de données NoSQL : MongoDB

J'ai choisi d'utiliser MongoDB pour stocker les préférences des chauffeurs, car cela représente plusieurs avantages par rapport à une base relationnelle classique.

En effet, les préférences peuvent varier d'un chauffeur à l'autre, certains champs sont communs (fumeurs, animaux), mais d'autres sont spécifiques et libres. Avec MongoDB, il est possible de stocker des documents JSON dynamiques, sans imposer un schéma rigide, ce qui permet d'enregistrer facilement un objet de préférences évolutif, sans modifier la structure de base.

e. Déploiement : Heroku

J'ai choisi Heroku pour le déploiement de l'application ; les détails complets sont présentés dans le **chapitre 4 - Documentation de déploiement**.

En définitive, la stack technique choisie pour l'application Ecoride répond aux besoins spécifiques du projet. Elle assure à la fois performances, sécurité, accessibilité et évolutivité, tout en me permettant de consolider des compétences directement exploitables dans un environnement professionnel.

2. Configuration de l'environnement de travail

a. Environnement de conception et design

Figma

J'ai installé **Figma** dès les premières étapes du projet EcoRide pour concevoir les wireframes et les mockups, structurer visuellement les pages (accueil, recherche de trajets, espace utilisateur etc.).

C'est aussi sur Figma que j'ai regroupé les références graphiques et élaboré la charte visuelle ainsi qu'un moodboard pour assurer la cohérence du design.

J'ai intégré le plugin Able afin de vérifier les contrastes et la taille des textes, dans un souhait de respecter les recommandations RGAA.

Même si ce projet était individuel, utiliser Figma m'a permis de centraliser tous les éléments du projet, et de me former à un outil collaboratif indispensable pour travailler efficacement en équipe sur de futurs projets.

Lucidchart

J'ai utilisé **LucidChart** pour la modélisation de l'application. J'ai pu concevoir les **principaux diagrammes** UML nécessaires à la bonne compréhension du système, notamment : le diagramme de classes, le diagramme de cas d'utilisation, ainsi que le diagramme de séquence. J'ai également utilisé Lucidchart pour concevoir **l'arborescence de l'application**, afin de visualiser les liens entre les différentes pages. Cet outil m'a permis de clarifier l'architecture globale du projet avant de passer à l'étape des maquettes et du développement, et d'assurer une organisation logique et intuitive des contenus.

b. Développement local et gestion des versions

VS Code

J'ai choisi **VS Code** comme environnement principal de développement pour EcoRide, en l'enrichissant avec plusieurs extensions destinées à améliorer ma productivité et la qualité du code.

J'ai ainsi utilisé *PHP Intelephense* pour profiter d'une autocomplétion et d'une meilleure compréhension des classes et méthodes PHP, *PHP Getters & Setters* pour générer rapidement les accesseurs dans mes entités, *Twig Language 2* pour la coloration syntaxique des templates TWIG, *YAML* pour faciliter la lecture et l'édition des fichiers de configuration Symfony, ainsi que *dotENV* pour gérer proprement les variables d'environnement.

Côté qualité, *SonarQube* m'a permis d'analyser en temps réel la sécurité et la robustesse du code.

J'ai utilisé le terminal PowerShell intégré à VS Code pour exécuter toutes les commandes nécessaires au projet, que ce soit pour lancer le serveur Symfony, installer les dépendances, ou encore gérer entièrement le versionning Git, des commits aux push sur GitHub.

Pour mes prochains projets Symfony, j'installerai également les extensions *PHP Debug* et *PHP DocBlocker*, afin de pouvoir déboguer mon code pas à pas directement dans VS Code et générer automatiquement des commentaires PHPDoc pour documenter proprement mes classes et méthodes.

Git et GitHub

J'ai configuré **Git** localement pour gérer les versions du projet, en utilisant principalement le terminal PowerShell intégré à VS Code pour toutes les commandes Git (initialisation, commits, push, gestion des branches).

J'ai créé un dépôt **GitHub** public pour héberger le code source et suivre toutes les évolutions du projet. J'ai structuré les branches de manière classique avec une branche master pour les versions stables, une branche dev pour les développements courants, et des branches feature/... dédiées aux nouvelles fonctionnalités et fix/.. pour les corrections.

J'ai cependant commis quelques erreurs dans la gestion de mes branches, notamment l'oubli de pousser systématiquement la branche dev après fusion d'une

fonctionnalité, mais j'en prends note afin d'améliorer ma rigueur et appliquer un workflow Git plus strict sur mes prochains projets.

J'ai également mis en place un fichier **.gitignore** adapté à un projet Symfony, pour m'assurer que seuls les fichiers nécessaires au fonctionnement du projet soient versionnées, et éviter de compromettre des données sensibles.

c. Développement front-end et back-end

Bootstrap

J'ai installé Bootstrap via un **CDN**, en ajoutant simplement les liens `<link>` et `<script>` fournis par Bootstrap directement dans la balise `<head>` et avant la balise `</body>` de mes pages HTML.

Symfony

J'ai utilisé **Symfony**, le framework PHP, pour développer l'application EcoRide.

Tout d'abord, j'ai installé **XamPP**, un package regroupant PHP, Apache, MySQL, ce qui m'a permis d'avoir **PHP** installé et prêt à l'emploi sur mon poste.

J'ai ensuite installé **Composer**, le gestionnaire de dépendances PHP, tel que **Doctrine** pour l'ORM, **Twig** pour les templates, ou encore les outils liés aux tests et à la sécurité.

Enfin, j'ai installé le **Symfony CLI** pour la création et la gestion du projet Symfony. J'ai donc pu créer le projet EcoRide avec la commande :

```
symfony new EcoRide --version="7.2.x" --webapp
```

Cette commande a généré la structure complète du projet Symfony, avec les répertoires *src*, *templates*, *public*, etc.

Cette commande a lancé automatiquement Composer pour installer les dépendances PHP nécessaires, selon la version précisée.

Grâce à la commande `symfony server:start`, qui démarre un serveur interne fourni par Symfony, j'ai pu visualiser mon projet directement dans mon navigateur tout au long du développement.

d. Base de données

MySQL

J'ai choisi **MySQL** comme système de gestion de base de données relationnelle, installé via **XamPP**.

Couplé à Doctrine, MySQL permet de gérer les entités du projet (utilisateurs, trajets, réservations, avis etc.).

MongoDB

Pour utiliser **MongoDB** dans mon projet, j'ai choisi d'héberger ma base sur **MongoDB Atlas**. J'ai aussi utilisé **MongoDB Compass** pour visualiser et manipuler les collections directement depuis mon ordinateur.

Côté projet Symfony, j'ai installé via Composer le package PHP **mongodb/mongodb**, qui m'a permis de connecter mon code à la base NoSQL et de gérer les données des préférences des chauffeurs sous forme de documents JSON.

e. Sécurité, tests et qualité

SonarQube

Intégré à VS Code, SonarQube permet d'identifier rapidement les vulnérabilités et les mauvaises pratiques dans le code PHP et Javascript.

Mailtrap

J'ai utilisé Mailtrap pour simuler l'envoi des emails (annulation ou demande de validation de trajet) en environnement de développement, sans envoyer de vrais mails aux utilisateurs.

f. Veille technologique et perspectives

GitHub Actions

Je me documente actuellement sur GitHub Actions pour automatiser encore plus le processus CI/CD, notamment pour déclencher des tests unitaires et déployer automatiquement après validation.

Docker

De même, je prévois de recourir à Docker pour isoler et standardiser l'environnement d'exécution, ce qui facilitera encore plus la portabilité et la montée en charge du projet.

Tests et sécurité

Je souhaite également approfondir mes connaissances sur les tests (unitaires, fonctionnels et automatisés) ainsi que sur la sécurité en général. Mon objectif est de mieux anticiper les vulnérabilités (comme celles recensées par l'OWASP) et de mettre en place de bonnes pratiques dès les premières étapes du développement.

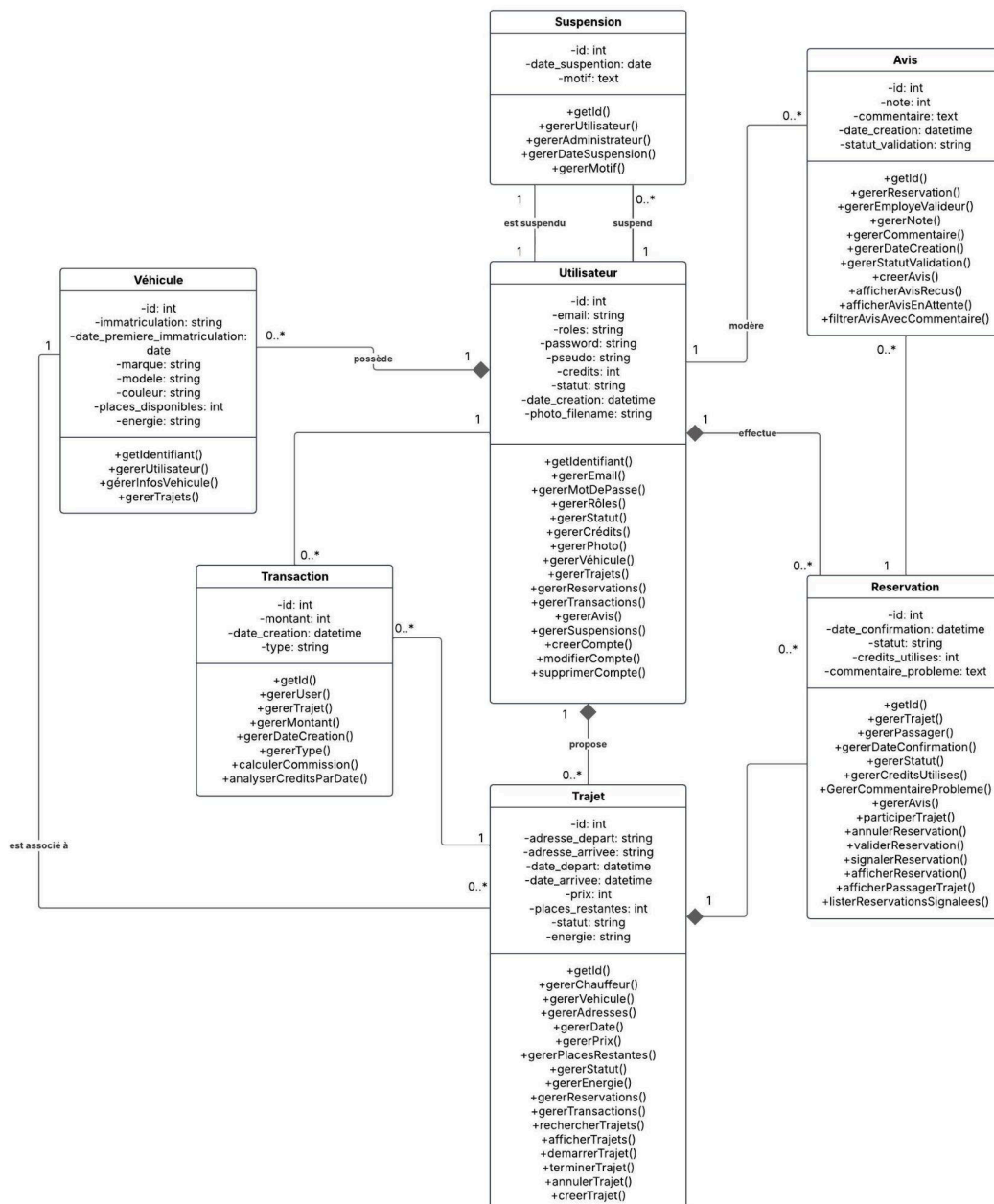
En résumé, cet environnement de travail m'a permis de réaliser le projet EcoRide dans des conditions proches du réel, avec un ensemble d'outils professionnels assurant productivité, qualité, sécurité et évolutivité.

3. Modélisation

Ce chapitre présente la modélisation de l'application à travers plusieurs diagrammes UML. Ces représentations graphiques permettent de mieux comprendre la structure, le fonctionnement et les interactions du système, avant de passer au développement. On y retrouve notamment le **diagramme de classes**, le **diagramme de cas d'utilisation** ainsi que le **diagramme de séquence**.

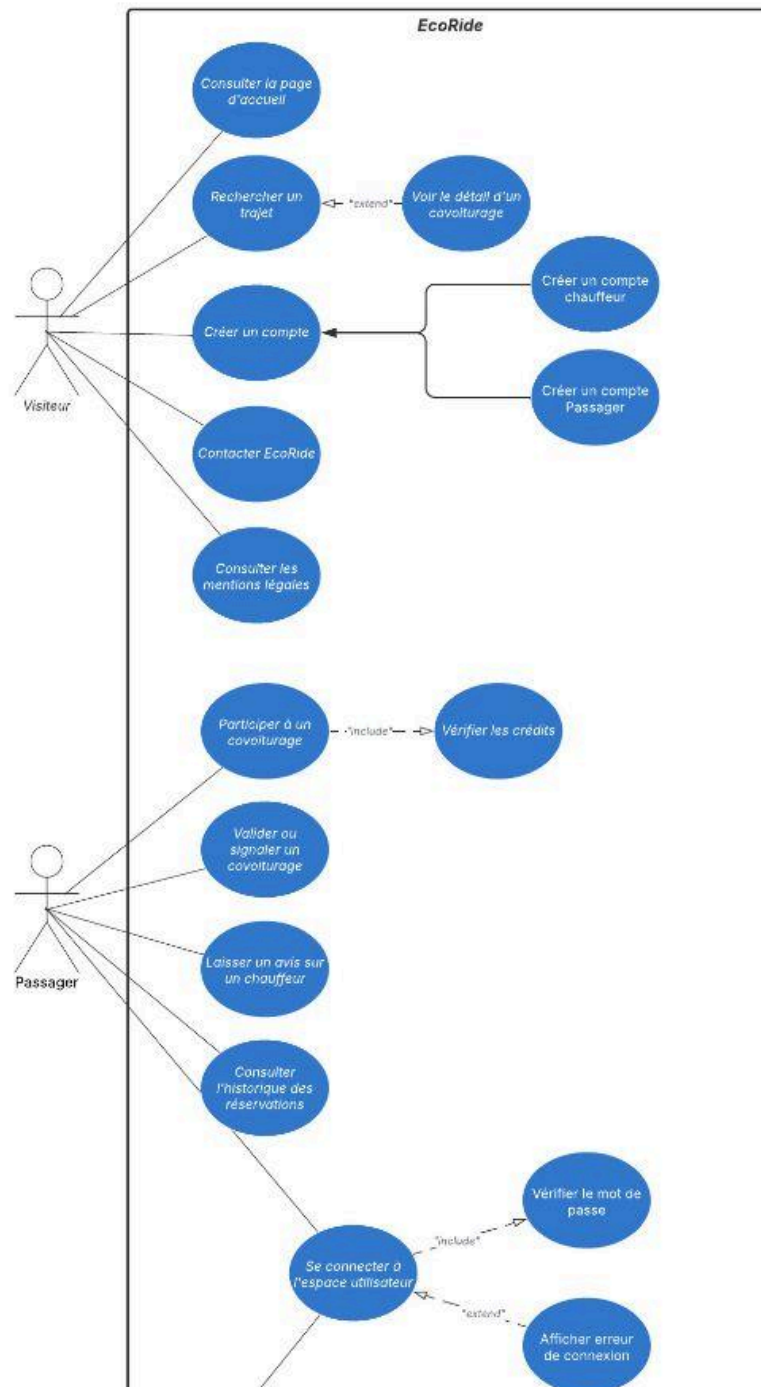
a. Diagramme de classe

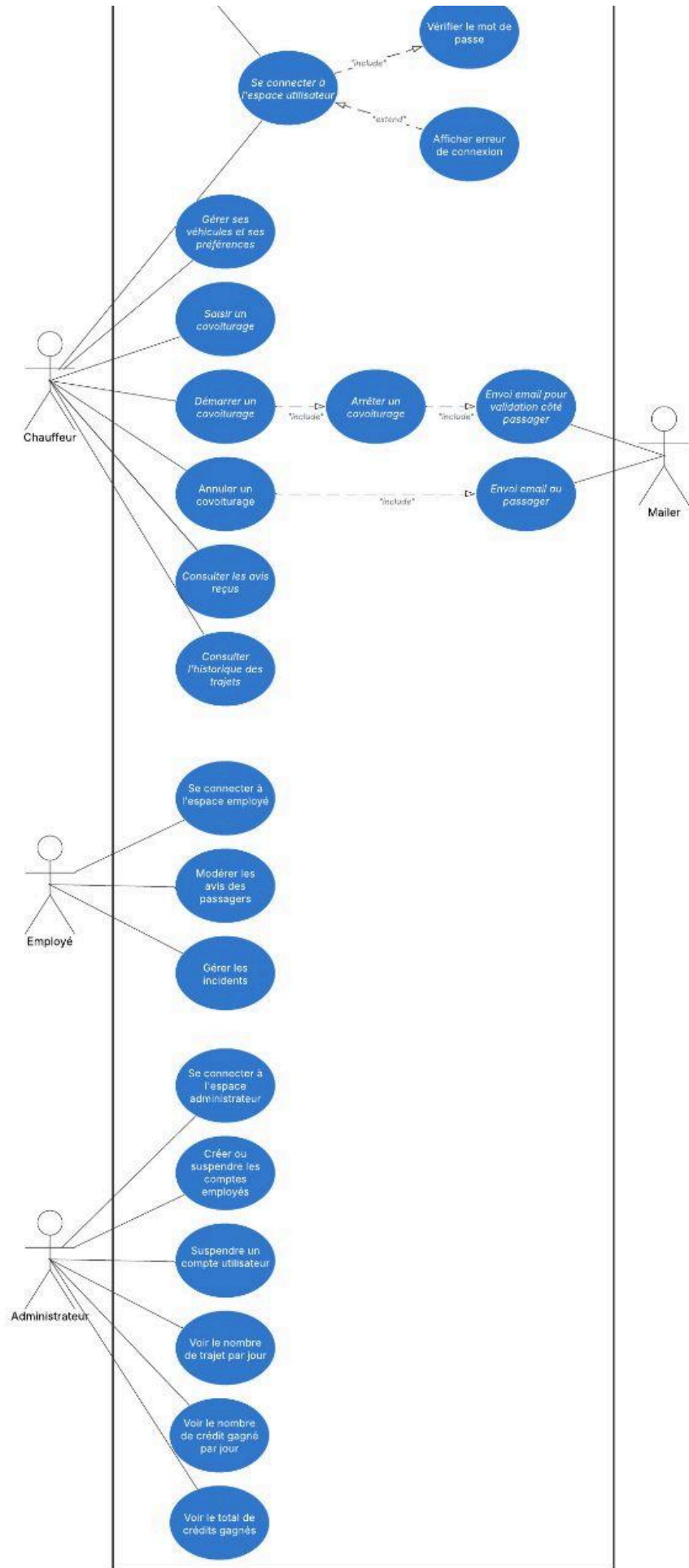
Voir le diagramme en haute définition [ici](#).



b. Diagramme de cas d'utilisation

Voir le diagramme en haute définition [ici](#).





c. Diagramme de séquence

Voir le diagramme en haute définition [ici](#).

Le diagramme de séquence présenté ci-dessous illustre la fonctionnalité **Participer à un trajet**, correspondant à l'**US 6** du projet EcoRide.

J'ai choisi de présenter cette fonctionnalité car elle représente une action centrale de l'application, mêlant à la fois logique métier, sécurité, gestion des crédits et persistance en base de données.

A noter que la vérification des places restantes est effectuée en amont lors de la recherche de trajets : seuls les trajets ayant encore des places disponibles sont proposés à l'utilisateur, ce qui permet de simplifier le traitement ici.

Rappel de l'US 6 :

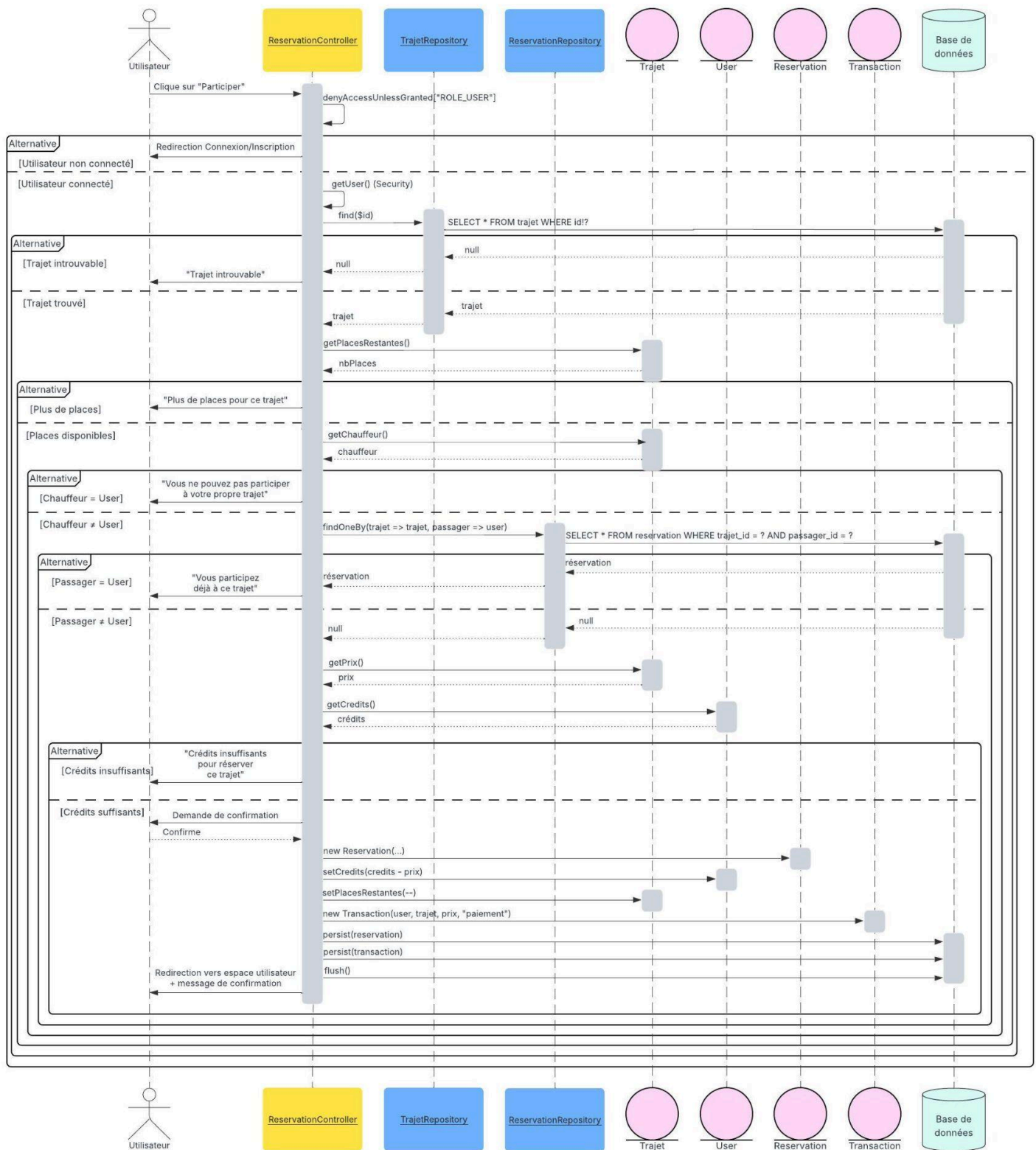
US 6 : Participer à un covoiturage

Utilisateur concerné : Visiteur, Utilisateur

Si le détail du voyage convient, il est possible de cliquer sur un bouton « participer », mais, uniquement s'il reste encore une place disponible et si l'utilisateur possède encore du crédit. De même, si la personne qui clic sur le bouton n'est pas « Utilisateur », alors, il lui sera proposé de se connecter / créer un compte.

Une fois le clic effectué par un Utilisateur, il doit être demandé une double confirmation afin de valider que l'utilisateur souhaite utiliser le nombre de crédit demandé. Cela ajoute à la suite de cela, un enregistrement de la confirmation dans l'espace de l'utilisateur « passager » (ainsi que la mise à jour des crédits) et met à jour le nombre de place dans le trajet de l'utilisateur « chauffeur ».

Diagramme de séquence - Fonctionnalité : Participer à un trajet



4. Documentation de déploiement

Pour mettre en production l'application EcoRide, j'ai choisi **Heroku**, une plateforme PaaS qui m'a permis de déployer mon application rapidement et en toute simplicité, sans avoir à administrer moi-même un serveur, tout en garantissant un accès HTTPS sécurisé.

Avant toute chose, j'ai finalisé le développement local de l'application sur ma machine et utilisé **Git** pour versionner le projet. Une fois les fonctionnalités terminées et testées, j'ai procédé à un merge de la branche de développement vers la branche principale (master), afin d'avoir un code stable et prêt à être déployé.

J'ai ensuite créé un compte sur **Heroku**, puis exécuté la commande `heroku login` pour m'authentifier depuis mon terminal (PowerShell intégré à VS Code), et créé mon application EcoRide avec la commande `heroku create ecoride-app`, ce qui a généré automatiquement une URL Heroku (<https://ecoride-app.herokuapp.com>)

Pour permettre à Heroku de savoir comment lancer mon projet Symfony, j'ai créé un fichier **Procfile** à la racine du projet, contenant :

```
web : heroku-php-apache2 public/
```

Ce fichier indique à Heroku d'utiliser Apache pour servir les fichiers à partir du répertoire `public/`, conformément à la structure de Symfony.

J'ai ensuite configuré sur Heroku la variable d'environnement **APP_ENV=prod** grâce à la commande `heroku config:set APP_ENV=prod`, pour indiquer à Symfony de fonctionner en mode production. Cela permet d'optimiser les performances et d'éviter d'exposer les informations sensibles en cas d'erreur.

J'ai également configuré la variable d'environnement **APP_SECRET** sur Heroku via `heroku config:set APP_SECRET="..."`. Cette variable sert à générer les tokens CSRF et à sécuriser les sessions dans Symfony. Cela permet de

garantir l'intégrité des données sensibles et d'éviter d'exposer cette clé secrète dans le code source.

Heroku n'intègre pas MySQL par défaut. Pour héberger la base de données relationnelle, j'ai donc activé l'add-on **JawsDB** avec la commande : `heroku addons:create jawsdb:kitefin`, ce qui a généré une URL de connexion **JAWSDB_URL** visible dans les variables d'environnement Heroku.

Pour que Symfony (et Doctrine) l'utilise, j'ai configuré Heroku pour que cette URL devienne la **DATABASE_URL** attendue par symfony, avec la commande : `heroku config:set DATABASE_URL=$JAWSDB_URL`.

J'ai ensuite effectué les migrations Doctrine vers la base JawsDB, ce qui a permis de créer les tables nécessaires directement sur le serveur MySQL cloud.

Pour la partie NoSQL, j'ai utilisé **MongoDB Atlas**. J'ai d'abord configuré les règles réseau dans le dashboard Atlas pour autoriser les connexions publiques (0.0.0.0/0), ce qui inclut les IP publiques des serveurs Heroku. Cela garantit que les requêtes provenant de mon application déployée peuvent atteindre la base MongoDB, tout en laissant la possibilité de restreindre plus finement cet accès à l'avenir.

Puis j'ai récupéré la chaîne de connexion que j'ai définie dans les variables d'environnement Heroku grâce à la commande `heroku config:set MONGODB_URL="mongodb+srv://..."`.

Enfin, comme j'ai installé via Composer le client officiel MongoDB PHP, mon application peut automatiquement utiliser la variable **MONGODB_URL** pour se connecter à MongoDB Atlas et accéder aux collections NoSQL.

J'ai également configuré sur Heroku la variable d'environnement **MAILER_DSN**, via `heroku config:set MAILER_DSN="smtp://..."`, ce qui permet à Symfony d'utiliser son composant Mailer pour envoyer des mails depuis l'application. Cela centralise la configuration, sécurise les identifiants du service SMTP et garantit le bon fonctionnement de l'envoi des mails en production.

Une fois toutes les configurations terminées, j'ai déployé le projet sur Heroku via la commande : `git push heroku master`.

Heroku a alors exécuté composer install, configuré l'application selon les variables d'environnement, et lancé le projet en production.

Par la suite, pour chaque mise à jour du code, il me suffit de faire :

```
git add .
```

```
git commit -m "message"
```

```
git push heroku master
```

Heroku récupère ainsi les modifications et redéploie automatiquement l'application.

Pour rendre EcoRide accessible via mon propre nom de domaine et non plus via l'adresse fournie par Heroku, j'ai acheté un nom de domaine chez **Gandi** : **ecoride-app.fr** et configuré la zone DNS pour qu'il pointe vers mon application Heroku.

J'ai ajouté un enregistrement **CNAME** pour `www`, qui redirige **www.ecoride-app.fr** vers mon application Heroku et un **ALIAS** pour `@`, afin que **ecoride-app.fr** (sans `www`) pointe également vers Heroku.

Grâce à cette configuration, mon application est accessible depuis mon domaine personnalisé, que l'utilisateur saisisse l'URL avec ou sans `www`.

Pour assurer la cohérence des URLs et forcer l'accès sécurisé, j'ai implémenté un **EventSubscriber** dans mon code Symfony qui redirige automatiquement toutes les requêtes vers **https://www.ecoride-app.fr**.

Ainsi, même si l'utilisateur saisit l'adresse sans `www` ou sans `https`, il est systématiquement redirigé vers l'URL canonique, ce qui assure une connexion HTTPS sécurisée, conforme aux recommandations RGPD.

Heroku gère automatiquement la sécurisation HTTPS via des certificats Let's Encrypt, ce qui m'a permis d'obtenir un site accessible en HTTPS sans avoir à configurer manuellement de certificat SSL.

Cette démarche de déploiement m'a permis de mettre EcoRide en production dans des conditions proches de celles d'un projet professionnel, avec un

hébergement, des bases de données sécurisées, des variables d'environnement pour protéger les informations sensibles et un domaine personnalisé accessible en HTTPS. Cela garantit à la fois la qualité, la sécurité et l'évolutivité du projet.

Conclusion

Le développement de l'application EcoRide m'a permis de mettre en œuvre l'ensemble des compétences attendues d'un développeur web : de la compréhension du besoin métier jusqu'à la mise en production de l'application.

Ce projet a été l'occasion de concevoir une architecture technique, de structurer le code selon les bonnes pratiques, et d'assurer la cohérence entre le front-end, la logique métier et la persistance de données.

La réalisation de ce projet m'a permis de progresser : j'ai appris à gérer toutes les étapes d'un cycle de développement web, à prendre des décisions techniques, à résoudre des problèmes concrets et à documenter proprement mon travail.

A travers cette documentation, j'ai souhaité rendre compte de la démarche technique suivie, des choix réalisés et de la rigueur apportée à chaque étape du projet. Elle constitue une trace professionnelle de ma capacité à travailler sur un projet web complet, en autonomie et dans un cadre réaliste.